

CLARYXA

Complete Development Guide

From Frontend Shells to a Fully Wired AI Learning Platform

Solo Developer

Step-by-Step

Production Ready

This guide covers every phase of building Claryxa — from setting up your infrastructure to wiring your AI pipeline. Follow each phase in order. Do not skip ahead.

Table of Contents

Phase 1	Supabase Setup	Infrastructure foundation — auth, DB, storage
Phase 2	FastAPI Backend Skeleton	Project structure, environment, CORS
Phase 3	Authentication	Signup, login, JWT middleware, security
Phase 4	Connect Frontend Auth	Wire login/signup pages to real auth
Phase 5	Database Schema	Design all tables with RLS policies
Phase 6	File Upload & Storage	Upload endpoints, validation, metadata
Phase 7	File Viewing	PDF conversion, PDF.js viewer, signed URLs
Phase 8	RAG Pipeline	Text extraction, chunking, embeddings, ChromaDB
Phase 9	AI Integration	Gemini/OpenAI setup, prompt building, chat endpoint
Phase 10	Security Hardening	Rate limiting, API key protection, input validation
Phase 11	Tasks & Calendar	CRUD endpoints, frontend wiring
Phase 12	Analytics Dashboard	Study time tracking, progress display

Supabase is your entire data backbone — authentication, database, and file storage in one place. This must be set up before writing a single backend line.

Getting Started

- Go to supabase.com and create a new project. Choose a strong database password and save it.
- Go to Authentication → Settings. Enable Email/Password sign in. Disable phone, OAuth for now.
- Go to Storage → Create a new bucket called **documents**. Set it to **Private**.
- Go to Project Settings → API. Copy your Project URL, anon public key, and service role key.

Row Level Security (RLS)

Enable RLS on every table you create. This ensures users can only access their own data — even if your backend has a bug, Supabase will not return another user's data. This is not optional.

Storage Folder Structure

```
documents/  
  {user_id}/  
    original/    ← user uploaded files  
    processed/   ← converted PDFs
```

End of Phase 1 Checklist

- Supabase project created
- Email/password auth enabled
- Documents storage bucket created (private)
- Project URL and API keys saved safely
- RLS enabled on all tables

Set up your backend project structure. At this stage nothing needs to work — just get the server running cleanly.

Project Structure

```
backend/  
  main.py  
  .env          ← ALL keys live here only  
  requirements.txt  
  routes/  
    auth.py  
    documents.py  
    chat.py  
    tasks.py  
    analytics.py  
  models/  
    user.py  
    document.py  
    task.py  
  services/  
    supabase_client.py  
    storage_service.py  
    ai_service.py  
    extraction_service.py  
    vector_service.py
```

.env File Contents

```
SUPABASE_URL=your_project_url  
SUPABASE_ANON_KEY=your_anon_key  
SUPABASE_SERVICE_KEY=your_service_role_key  
GEMINI_API_KEY=your_gemini_key  
OPENAI_API_KEY=your_openai_key (if using OpenAI)
```

Required Packages

```
fastapi
uvicorn
python-dotenv
supabase
python-multipart
slowapi
pymupdf
python-docx
python-pptx
chromadb
langchain
google-generativeai
```

Run the server locally:

```
uvicorn main:app --reload
```

If it starts without errors, Phase 2 is done. Routes can return dummy data for now.

End of Phase 2 Checklist

- Backend folder structure created
- .env file created with all keys (never commit this to git)
- All packages installed and in requirements.txt
- Server runs locally without errors
- .gitignore includes .env

Authentication is the most critical phase. Every other feature depends on knowing who the user is. Do not move on until this is fully tested.

Backend Endpoints to Build

POST /auth/signup

Takes email + password. Creates user in Supabase. Returns session token.

POST /auth/login

Takes email + password. Verifies with Supabase. Returns JWT token.

POST /auth/logout

Invalidates the current session.

Auth Middleware

Build a middleware function that runs before every protected route. It must:

- Read the **Authorization: Bearer <token>** header from the request
- Verify the JWT token with Supabase
- Extract the `user_id` and attach it to the request
- Return a 401 error immediately if token is missing or invalid
- Every route except `/auth/login` and `/auth/signup` must use this middleware

Frontend Requirements

- Store JWT token in memory or httpOnly cookie — NOT localStorage (vulnerable to XSS)
- Attach token to every API request in the Authorization header
- If any request returns 401, redirect user to login page immediately
- Clear the token completely on logout

Testing Auth (Do This Before Moving On)

Test: Sign up → Log in → Make a protected request (should work) → Log out → Make the same request again (should return 401). If all four work correctly, auth is done.

End of Phase 3 Checklist

- Signup endpoint working
- Login endpoint returns JWT token
- Auth middleware protects all routes

- 401 returned on invalid/missing token
- Token storage is secure (not localStorage)
- Logout clears session

This is your first real wire. The login and signup pages stop being decorative and start doing real things.

What to Wire Up

- Signup form submit button → calls POST /auth/signup with email + password
- Login form submit button → calls POST /auth/login, saves token, redirects to dashboard
- Add loading state while request is in flight (disable button, show spinner)
- Add proper error messages: 'Invalid credentials', 'Email already exists' — not just 'Error'
- Protect all frontend routes — if user is not logged in and visits /dashboard, redirect to /login
- Add logout button on dashboard → calls POST /auth/logout, clears token, redirects to login

At the end of this phase, a real user can create an account, log in, and land on the dashboard. Other pages are still empty but they are now behind a real auth wall.

End of Phase 4 Checklist

- Signup form calls real API endpoint
- Login form calls real API endpoint and stores token
- Dashboard redirects unauthenticated users to login
- Error states shown to user on failed login/signup
- Loading states during API calls
- Logout works end-to-end

Design your database tables before writing any endpoints. All tables go in Supabase → SQL Editor.

Documents Table

```
create table documents (  
  id uuid default gen_random_uuid() primary key,  
  user_id uuid references auth.users(id) on delete cascade,  
  name text not null,  
  original_name text,  
  file_type text,  
  file_path text,  
  processed_path text,  
  size_bytes bigint,  
  processing_status text default 'pending',  
  uploaded_at timestamp default now()  
);
```

Tasks Table

```
create table tasks (  
  id uuid default gen_random_uuid() primary key,  
  user_id uuid references auth.users(id) on delete cascade,  
  title text not null,  
  description text,  
  due_date date,  
  status text default 'pending',  
  created_at timestamp default now()  
);
```

Study Sessions Table

```
create table study_sessions (  
  id uuid default gen_random_uuid() primary key,  
  user_id uuid references auth.users(id) on delete cascade,  
  date date default current_date,  
  seconds_spent integer default 0  
);
```

RLS Policies (Apply to All Tables)

```
-- Example for documents table (repeat for tasks, study_sessions)
alter table documents enable row level security;

create policy 'Users can view own documents'
on documents for select
using (auth.uid() = user_id);

create policy 'Users can insert own documents'
on documents for insert
with check (auth.uid() = user_id);

create policy 'Users can delete own documents'
on documents for delete
using (auth.uid() = user_id);
```

End of Phase 5 Checklist

- documents table created
- tasks table created
- study_sessions table created
- RLS enabled on all tables
- RLS policies for select, insert, update, delete on each table

Build the file upload system. Every uploaded file must be tied to a `user_id` so users only see their own files.

Upload Endpoint — What It Must Do (In Order)

- 1 Verify the user is authenticated (middleware handles this automatically)
- 2 Receive the uploaded file from the request body
- 3 Validate file type — only allow PDF, DOCX, PPTX, TXT. Reject everything else with 400 error
- 4 Check file size — reject files over 20MB with a clear error message
- 5 Generate a unique filename using UUID to avoid collisions
- 6 Upload to Supabase Storage under path: `{user_id}/original/{unique_filename}`
- 7 Save file metadata to documents table (name, type, path, size, `user_id`)
- 8 Trigger background processing (extraction + embedding) — do not make user wait
- 9 Return the new document record to the frontend immediately

Other Endpoints to Build

GET <code>/documents</code>	Returns all documents for the logged-in user
GET <code>/documents/{id}</code>	Returns a single document's details and signed URL
DELETE <code>/documents/{id}</code>	Deletes file from storage and record from DB

Processing Status

The `processing_status` column tracks where each document is in the pipeline:

pending	Just uploaded, processing not started yet
processing	Actively being converted/extracted/embedded
ready	AI processing complete, can be used for chat
failed	Something went wrong during processing

End of Phase 6 Checklist

- POST `/documents/upload` endpoint working

- File type validation (reject non PDF/DOCX/PPTX/TXT)
- File size limit enforced (20MB max)
- Files stored in Supabase Storage under user_id folder
- Metadata saved to documents table
- Background processing triggered on upload
- GET /documents returns user's files only
- DELETE /documents/{id} removes from storage and DB
- Library page wired to show real uploaded files

Convert all uploaded files to PDF for consistent display. One viewer handles everything — PDF, DOCX, PPTX, TXT.

Why Convert Everything to PDF

- One viewer component handles all file types
- One text extraction method for the entire AI pipeline
- Consistent rendering across all devices and browsers
- Far simpler codebase — no special handling per file type

Conversion Pipeline

```
User uploads file
↓
If PDF already → skip conversion
If DOCX/PPTX/TXT → LibreOffice conversion
↓
Store original at: {user_id}/original/{filename}
Store PDF at:      {user_id}/processed/{filename}.pdf
↓
Extract text from PDF → chunk → embed
```

LibreOffice Conversion (Backend)

Install LibreOffice on your backend server:

```
sudo apt install libreoffice
```

Trigger conversion via Python subprocess:

```
import subprocess

subprocess.run([
    "libreoffice", "--headless",
    "--convert-to", "pdf",
    "--outdir", output_folder,
    input_file_path
])
```

PDF Viewer in Frontend

Use react-pdf (wraps PDF.js — the industry standard):

```
npm install react-pdf
```

Viewer Flow

```
graph TD
    A[User clicks document in Library] --> B[Frontend calls GET /documents/{id}]
    B --> C[Backend generates signed URL from Supabase Storage  
(expires in 1 hour – never use public URLs)]
    C --> D[react-pdf renders the PDF from the signed URL]
    D --> E[User can scroll, zoom, and read]
```

Always use signed URLs, never make your storage bucket public. A signed URL is temporary and tied to the requesting user — it expires automatically.

End of Phase 7 Checklist

- LibreOffice installed on backend server
- Conversion service converts DOCX/PPTX/TXT to PDF
- Both original and processed PDF stored in Supabase
- react-pdf installed in frontend
- Document viewer page uses signed URLs
- Signed URLs expire after 1 hour
- Viewer shows document name and page controls

RAG (Retrieval-Augmented Generation) is what makes Claryxa actually read your documents. Without it, the AI just makes up answers. This is the most complex phase — take it one step at a time.

The Full Pipeline

```
PDF uploaded
  ↓ Extract raw text (PyMuPDF)
  ↓ Split into chunks (~500 tokens, 50-token overlap)
  ↓ Convert chunks to embeddings (vectors)
  ↓ Store embeddings in ChromaDB

User asks question
  ↓ Convert question to embedding
  ↓ Search ChromaDB for similar chunks
  ↓ Retrieve top 3-5 matching chunks
  ↓ Build prompt (chunks + question)
  ↓ Send to AI API
  ↓ Return answer to frontend
```

Step 8A — Text Extraction Service

Build `extraction_service.py`. It must handle all file types:

PDF	PyMuPDF — extract text page by page
DOCX	python-docx — extract paragraph by paragraph
PPTX	python-pptx — extract text from each slide
TXT	Plain file read — no library needed

Important: If PyMuPDF returns empty text on a PDF, it is a scanned image-based PDF. Detect this and show the user a clear error: 'This PDF is image-based and cannot be processed for AI.'

Step 8B — Text Chunking

Build `chunking_service.py` with these rules:

- Split text into chunks of ~500 tokens (roughly 400 words)
- Use overlapping chunks — each chunk shares 50-100 tokens with the next
- Store metadata with each chunk: `document_id`, `chunk_index`, `page_number`

- Overlap prevents answers from being cut off at chunk boundaries

```
Chunk 1:  tokens 0    → 500
Chunk 2:  tokens 400 → 900   (100 token overlap)
Chunk 3:  tokens 800 → 1300  (100 token overlap)
...
```

Step 8C — Embeddings + ChromaDB

An embedding converts text into a list of numbers representing its meaning. Similar meanings = similar numbers = close in vector space. ChromaDB stores and searches these vectors.

- Use Google's text-embedding-004 model (free tier, pairs well with Gemini)
- Or OpenAI's text-embedding-3-small (very cheap, excellent quality)
- Organize ChromaDB collections per user — never mix users' document chunks
- Tag every entry with `user_id` in metadata as an extra safety layer

Step 8D — Background Processing on Upload

Use FastAPI BackgroundTasks so the user doesn't wait for processing:

```
1. Upload endpoint receives file
2. Save to storage immediately
3. Save metadata with status = 'processing'
4. Return success to frontend instantly
5. Background task runs: extract → chunk → embed → store
6. Update status to 'ready' when done (or 'failed' on error)
```

End of Phase 8 Checklist

- `extraction_service.py` handles PDF, DOCX, PPTX, TXT
- Scanned PDF detection with user-friendly error
- `chunking_service.py` with 500 token chunks and overlap
- ChromaDB set up locally
- Embeddings generated and stored per document
- Collections organized by `user_id`
- Background processing triggered on upload
- `processing_status` updates correctly (pending → processing → ready/failed)

Connect Gemini (recommended) or OpenAI to power the chat. The AI never reads your PDFs directly — your RAG pipeline feeds it the relevant sections.

Recommended Stack

AI Chat / Q&A;	Gemini 1.5 Flash — fast, free tier, already in your ecosystem
Embeddings	Google text-embedding-004 — free tier, pairs with Gemini
Alternative AI	OpenAI gpt-4o-mini — cheap, excellent quality, best docs
Alt Embeddings	OpenAI text-embedding-3-small — extremely cheap
Local (Dev Only)	Ollama + LLaMA 3 — free but slow without GPU

The Chat Endpoint — POST /chat/ask

```
Receive: question + document_id (optional)
  ↓
Auth check (middleware)
  ↓
Convert question to embedding
  ↓
Search ChromaDB (filtered by user_id)
  ↓
Retrieve top 3-5 chunks
  ↓
Build prompt (see below)
  ↓
Call Gemini/OpenAI API
  ↓
Return answer + source document name
```

Prompt Structure

The instruction 'based ONLY on the provided content' is critical. Without it the AI blends your document with its training data and starts hallucinating.

```
You are a study assistant. Answer the user's question
based ONLY on the provided document content.
If the answer is not in the content, say so honestly.
Do not make up or add information not in the text.
```

```
Document content:
```

```
[chunk 1 text]
```

```
[chunk 2 text]
```

```
[chunk 3 text]
```

```
User question:
```

```
[question here]
```

```
Answer:
```

Frontend Chat Page Wiring

- Document selector dropdown → fetches from GET /documents, lets user pick which doc to chat with
- If no document selected, search across all user's documents
- Message input + send → calls POST /chat/ask with question and document_id
- Show typing indicator while waiting for response
- Display answer in chat bubble with source label: 'Sourced from: lecture1.pdf'
- If document processing_status is not 'ready', disable send button and show 'Document still processing...'

End of Phase 9 Checklist

- Gemini or OpenAI API key in .env
- ai_service.py builds prompt and calls API
- POST /chat/ask endpoint working end-to-end
- Answers sourced from real document content
- Source document name returned with answer
- Chat page wired to real backend
- Typing indicator during AI response
- Disabled state when document is still processing

Security is not optional. These measures protect your users, your API keys, and your wallet.

Rate Limiting (slowapi)

- Limit AI chat endpoint to 20 requests per user per hour — AI calls cost money
- Limit upload endpoint to 10 files per user per hour
- General API rate limit: 100 requests per IP per minute
- Return HTTP 429 (Too Many Requests) with a clear message when limit is hit

API Key Protection

- API keys live ONLY in .env on the server — never in frontend code
- Never return API keys in any API response
- Never log API keys in console output or error logs
- Add .env to .gitignore before your very first git commit
- Rotate your key immediately if you suspect it was exposed
- Set a hard spending limit on your Gemini/OpenAI account dashboard

Input Validation

- Max question length: 1000 characters — reject longer inputs with 400 error
- File type whitelist on upload — check actual MIME type, not just extension
- File size limit: 20MB max
- Sanitize all text inputs before storing in database
- Validate document_id belongs to the requesting user before using it

Prompt Injection Protection

Users can try to hijack your AI by injecting instructions into their questions. Add a basic filter that rejects questions containing phrases like:

```
"ignore previous instructions"  
"you are now"  
"forget everything"  
"disregard your instructions"  
"act as"
```

Cost Monitoring

- Log every AI API call: user_id, timestamp, input tokens, output tokens

- This lets you track costs per user and spot abuse patterns
- Set alerts on your Gemini/OpenAI dashboard for unusual spending

End of Phase 10 Checklist

- Rate limiting on all AI endpoints
- Rate limiting on upload endpoint
- .env in .gitignore from day one
- Hard spending limit set on AI provider dashboard
- Input length validation on all endpoints
- File type MIME validation on upload
- Prompt injection basic filter
- AI call logging (user_id + token counts)

Tasks and Calendar are pure CRUD — no AI needed. These are the simplest features to build once your infrastructure is in place.

Tasks Endpoints

GET /tasks	Return all tasks for the logged-in user
POST /tasks	Create a new task (title, description, due_date, status)
PATCH /tasks/{id}	Update task (mark complete, change due date, edit title)
DELETE /tasks/{id}	Delete a task

Calendar Notes

- Calendar is just tasks displayed by date — no separate table needed
- Fetch tasks with due_date in the current month for calendar view
- Tasks page and Calendar page share the same backend endpoints
- Use a frontend calendar library like react-calendar or FullCalendar

End of Phase 11 Checklist

- All 4 task CRUD endpoints working
- Tasks page wired to real API
- Create task form functional
- Mark as complete works
- Delete task works
- Calendar page shows tasks by due date

Track study time and display progress on the dashboard. You need three things: detect activity, measure time, store and display it.

Frontend — Activity Detection

Track user activity in the browser. Count time only when the user is actually active:

```
Track events: mousemove, click, scroll, keypress

let lastActivity = Date.now()

onUserActivity() {
  lastActivity = Date.now()
}

Every 10 seconds:
  if (Date.now() - lastActivity < 5 minutes):
    studyTimeToday += 10 seconds
    send to backend: POST /analytics/study-time { seconds: 10 }
```

Backend Endpoints

POST /analytics/study-time	Receives time increment, adds to today's record for user
GET /analytics/summary	Returns today's time, weekly data, streak count

Backend Logic for Study Time

```
Receive time increment (e.g. 10 seconds)
↓
Find today's study_sessions row for this user
↓
If row exists → add seconds to seconds_spent
If no row → create new row for today
↓
Save updated total
```

Dashboard Display

- Today's study time — convert seconds to hours and minutes (e.g. '1h 24m')

- Progress bar toward daily goal (e.g. 120 minutes default goal)
- Weekly graph showing past 7 days — use Recharts or Chart.js
- Documents studied count — count distinct `document_ids` from chat history
- Study streak — count consecutive days with `study_sessions > 0`

End of Phase 12 Checklist

- Frontend activity detection working
- Study time sent to backend every 10 seconds
- `study_sessions` table updating correctly
- `GET /analytics/summary` returns correct data
- Dashboard shows real study time
- Weekly graph displays past 7 days
- Daily goal progress bar working

Complete Build Summary

Phase	What You Build	
Phase 1	Supabase Setup	✓
Phase 2	FastAPI Backend Skeleton	✓
Phase 3	Authentication (JWT + Middleware)	✓
Phase 4	Frontend Auth Wiring	✓
Phase 5	Database Schema + RLS	✓
Phase 6	File Upload + Storage	✓
Phase 7	File Viewing (PDF Conversion + PDF.js)	✓
Phase 8	RAG Pipeline (Extract → Chunk → Embed → Query)	✓
Phase 9	AI Chat Integration (Gemini / OpenAI)	✓
Phase 10	Security Hardening (Rate Limit, Validation, Keys)	✓
Phase 11	Tasks & Calendar (CRUD)	✓
Phase 12	Analytics Dashboard (Study Time + Progress)	✓

Follow each phase in order. Do not skip ahead to AI features before auth and storage are solid. Use Antigraity to generate code one step at a time — give it specific, scoped prompts. Test each phase before wiring the next one.

Recommended AI Provider

Chat & Q&A;	Gemini 1.5 Flash — free tier, fast, already in your ecosystem
Embeddings	Google text-embedding-004 — free tier

Backup option

OpenAI gpt-4o-mini + text-embedding-3-small

Good luck building Claryxa. The hardest part is starting — you've already done that.